SparseRL-Sync: Lossless Weight Synchronization with $\sim 100\times$ Less Communication

Lucas Hu*, Ranchi Zhao*, Isaac Zhu, Zach Zhang, Hscos Zhang, Hugh Yin, Jason Zhao†

Scitix

*Equal contribution. †Corresponding author.

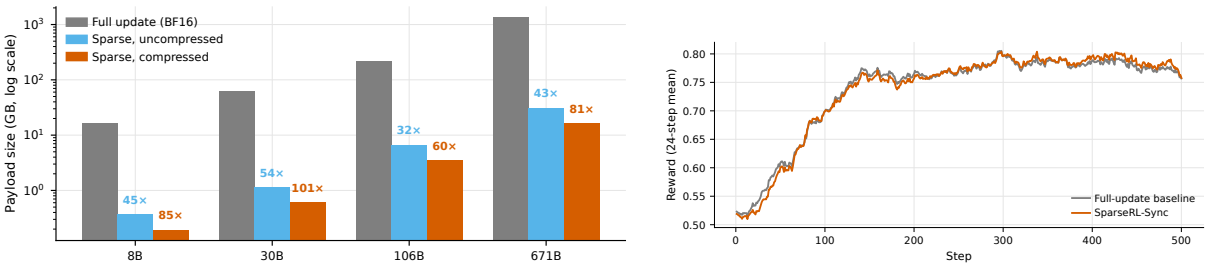
May 6, 2026

Abstract

In large-scale reinforcement learning (RL) systems with decoupled Trainer–Rollout execution, the Trainer must regularly synchronize policy weights to the Rollout side to limit policy staleness. When inter-node bandwidth is abundant, such synchronization is usually only a small fraction of end-to-end cost. As model size grows, however, the communication demand rises rapidly. In bandwidth-constrained or network-variable deployments—for example, cross-datacenter or cross-cluster settings, heterogeneous resource pools, and online RL—weight synchronization can become a dominant bottleneck for throughput and tail latency.

We observe that, in mainstream large-model RL training, the locations where parameters actually change are highly sparse at the element level (often 99%+ sparsity). Building on this observation, we propose and implement SparseRL-Sync, which replaces full-weight transfers with a *lossless* sparse update payload (indices and values) that can be *exactly* reconstructed on the inference side, thereby preserving 100% fidelity. Under a simplified cost model, sparse synchronization reduces the per-update communication volume from S to approximately S/X; with 99% sparsity ($X \approx 100$), this yields about a $100\times$ reduction in transmitted data. Combined with appropriate bucketing, SparseRL-Sync also reduces launch and control-plane overhead, significantly improving scalability and end-to-end efficiency in bandwidth-limited and highly asynchronous RL settings.

1 Introduction



(a) Per-synchronization payload across model scales: full update (BF16) vs. sparse (I, V) uncompressed vs. sparse (I, V) compressed. Sparse synchronization reduces the transfer by $32\times$–$54\times$ raw and $\approx 60\times$–$101\times$ after lossless compression (Section 3.3).

(b) Reward curves of the full-update baseline vs. SparseRL-Sync on Qwen3-30B-A3B-Instruct-2507 over 500 training steps. The two curves are nearly indistinguishable, confirming lossless fidelity.

Figure 1: Core result at a glance. SparseRL-Sync reduces the Trainer-to-Rollout weight-synchronization payload by $32\times$–$54\times$ raw and up to $\approx 100\times$ after lossless compression across model scales (left) while preserving training dynamics bit-exactly (right).

Codes will be released at https://github.com/scitix/helix.

Large-model RL training systems typically decouple the *Trainer* (training) from the *Rollout* (inference) component. The Trainer computes losses and updates model parameters from collected trajectories, while the Rollout uses the current policy to generate new trajectories. To limit policy staleness and preserve training stability, updated parameters must be synchronized regularly from Trainer to Rollout. This interaction can be understood from two complementary perspectives: the data exchanged between Trainer and Rollout, and the way the two components are deployed. We describe these two aspects in turn.

Two interaction flows. Trainer and Rollout interact through two distinct data flows.

- **Sample data flow:** Rollout performs inference and sampling, producing trajectories, tokens, and rewards that are returned to the Trainer. This is the primary training path and is typically less bandwidth-intensive than weight synchronization.
- **Weights data flow:** After one or more optimization steps, the Trainer synchronizes updated policy weights to Rollout. The payload size scales directly with model size, is typically much larger than the sample flow, and quickly becomes the bottleneck in bandwidth-constrained settings.

Two placement strategies. Trainer and Rollout are typically deployed in one of two ways.

- **Time-sharing:** The two components share the same set of GPUs and alternate between training and inference through time multiplexing. In this case, weight synchronization is typically limited to intra-process switching or intra-node transfer (e.g., CUDA IPC).
- **Space-sharing:** GPUs are partitioned between Trainer and Rollout so that training and inference can proceed concurrently. Weight updates must then be distributed across multiple GPUs or nodes, typically via collective communication (e.g., NCCL), making the system much more sensitive to network bandwidth and topology.

In large-model RL, Rollout's `generate()` is often the end-to-end throughput bottleneck and exhibits pronounced tail latency. To mitigate this bottleneck and improve overall GPU utilization, many systems adopt asynchronous RL (Async-RL), which is commonly implemented through space-sharing or fully disaggregated deployment to further decouple Trainer and Rollout. This design makes parameter synchronization a central scalability bottleneck.

This bottleneck is rooted in the way current systems perform Trainer-to-Rollout weight synchronization. Figure 2 (left column) illustrates the parameter-update pipeline in the open-source RL framework `slime` (THU-DCST, 2024). After a number of optimizer steps, the Trainer (i) gathers parameters along the TP and EP dimensions, (ii) performs format conversion and any necessary quantization, and (iii) broadcasts from Trainer to Rollout ranks.

The cost of this design becomes increasingly pronounced as model size grows and available bandwidth decreases. Figure 3 shows the estimated cost of performing a single parameter update for several representative models under different communication bandwidths. With high update frequency and large payloads, the cost grows almost linearly with model size and inversely with bandwidth, making weight synchronization a first-class scalability concern.

Existing engineering remedies. To make parameter synchronization work in practice, current systems combine a series of engineering techniques:

- **Bucketing and pipelining:** split the weight tensor into multiple buckets to reduce peak memory and shorten tail latency.
- **Communication–computation overlap:** hide synchronization behind the compute path, at the cost of additional thread/stream/scheduling complexity.

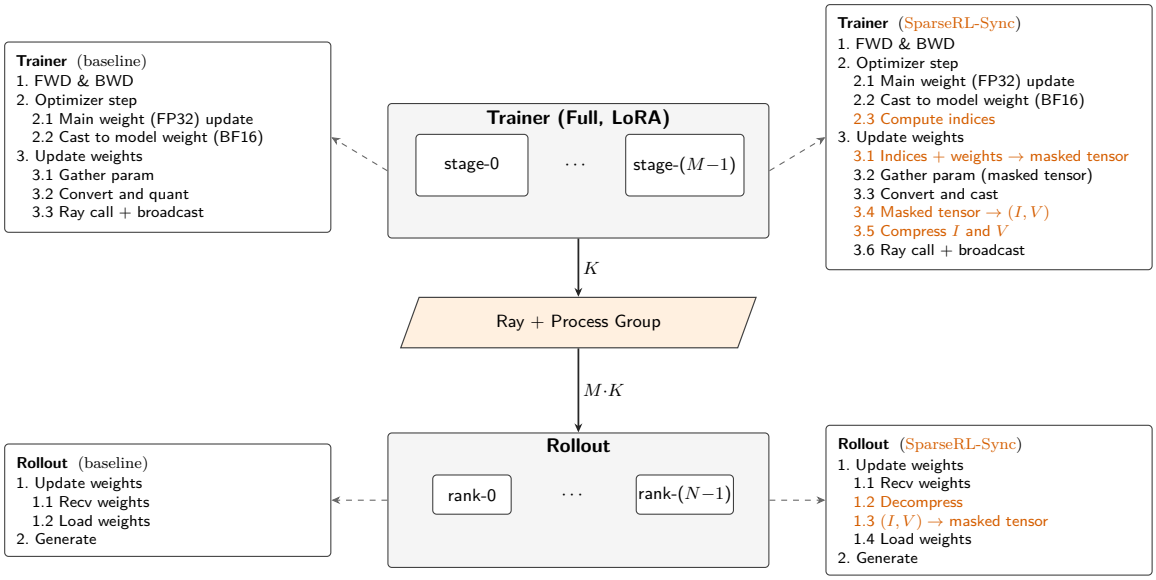


Figure 2: Trainer–Rollout weight-synchronization workflow. **Left column:** the full-update baseline pipeline used by open-source RL frameworks such as `slime`. **Right column:** the SparseRL-Sync pipeline, with newly inserted steps highlighted in vermillion. The center column shows the physical topology shared by both: M Trainer stages (PP size) $\rightarrow$ Ray + process group $\rightarrow$ N Rollout ranks. Each Trainer stage contributes K buckets to the broadcast, and the process group forwards the aggregated $M{\cdot}K$ buckets to every rank. SparseRL-Sync reduces each bucket's payload via sparse (I, V) encoding without altering the collective topology.

- **Eliminating redundant transfers:** avoid retransmitting identical weight versions, eliminate extra copies, and avoid amplification caused by merging-before-sending.
- **Bandwidth-aware scheduling:** when broadcasting to many Rollout nodes, schedule the distribution order and concurrency so that slow nodes do not stall the whole system.

These techniques mitigate synchronization overhead in important ways, but they do not change the communication object itself: the synchronized payload remains the full-weight tensor. As model size grows and deployment becomes increasingly bandwidth-constrained, this design choice remains a fundamental scalability bottleneck. Our work revisits this assumption. Rather than further optimizing the full-update synchronization path, we exploit the element-level sparsity of the BF16 weight delta sent from Trainer to Rollout and redesign synchronization around a lossless sparse update representation.

Contributions.

- We *extend the empirical scope* of the sparsity observation beyond prior RLVR results on a fixed model to a range of mainstream RL settings, including GRPO, DAPO, GSPO, asynchronous RL, and agentic RL, across dense and MoE models ranging from 8B to 671B parameters, and under BF16, FP16, and FP8 synchronization (Sections 2.1 and 4).
- We design SparseRL-Sync, a *lossless* sparse-synchronization mechanism for Trainer-to-Rollout weight updates, and develop a precise cost model for the sparse (I, V) payload together with lossless encoding schemes for both indices and values that lift the raw compression ratio of 32×–54× to a compressed ratio of ≈60×–101× across model scales from 8B to 671B (Sections 3 and 3.3).
- We validate SparseRL-Sync end-to-end in two complementary studies: a correctness study over 500 training steps confirms *bit-exact* equivalence to the full-update baseline

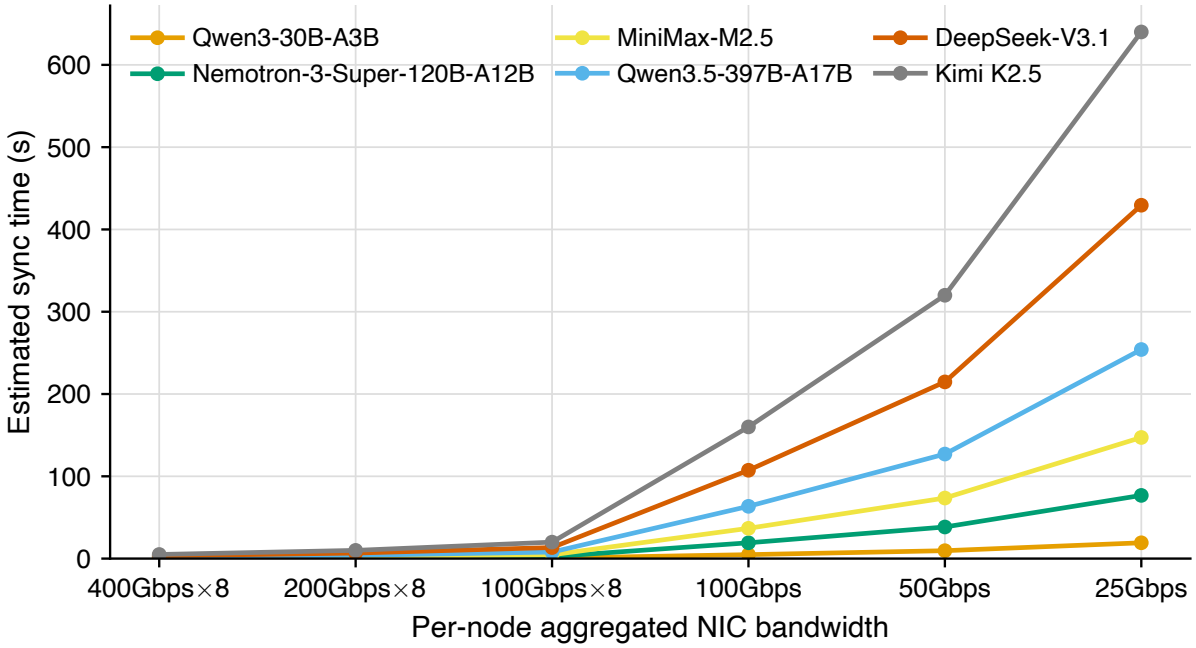


Figure 3: Estimated wall-clock cost of a single full-weight (BF16) parameter update for representative open models under different per-node aggregated NIC bandwidths, including Qwen3-30B-A3B (30B), Nemotron-3-Super-120B-A12B (120B), MiniMax-M2.5 (230B), Qwen3.5-397B-A17B (397B), DeepSeek-V3.1 (671B), and Kimi K2.5 (1TB). As model size increases and available bandwidth decreases, the synchronization cost rises sharply.

and an indistinguishable reward trajectory, and a performance study across two bandwidth regimes shows that sparse synchronization substantially reduces Trainer-to-Rollout broadcast time in both high- and low-bandwidth environments (Section 4).

2 Sparsity Analysis

The starting point of our system design is an empirical regularity: in mainstream large-model RL training, the BF16 model-weight delta sent to Rollout at every synchronization point is almost entirely sparse at the element level. Section 2.1 documents this regularity across five RL settings; Section 2.2 summarizes the explanation given by the Three-Gate Theory of Zhu et al. (2025) for why this happens. The theory itself is borrowed; our contributions are the empirical extension to MoE / GRPO (Shao et al., 2024) / DAPO (Yu et al., 2025) / GSPO (Zheng et al., 2025) / Async-RL / Agentic-RL settings and its system-level exploitation in Section 3.

2.1 Empirical Observations

Setup. We instrument Qwen3-30B-A3B-Instruct-2507 during RL fine-tuning and capture snapshots of the BF16 model weights immediately before and after each offline weight-update event. From these snapshots, we compute two statistics, both per tensor and aggregated across the model: (i) the fraction of elements whose BF16 value changes (*element-level update ratio*, with sparsity defined as its complement), and (ii) the fraction of tensors for which no element changes (*tensor-level inactive ratio*). We use BF16 for both training and inference, and otherwise follow the default configuration of `slime`. We repeat this measurement under five RL settings: GRPO, DAPO, GSPO, an asynchronous RL variant (Async-RL), and an agentic RL variant (Agentic-RL).

Table 1: Comparison of sparse-synchronization support and reported performance across recent RL training systems. "Validated size" is the largest model scale for which weight synchronization has been publicly reported; "Perf." is the reported synchronization latency at that scale. "Validated in Agentic-RL" indicates whether sparse synchronization has been validated specifically under agentic RL workloads. Performance figures for third-party systems are taken from publicly available technical reports, blog posts, or official documentation and are reproduced here for reference only; figures marked "?" were not publicly disclosed at the time of writing.

System	Sparse sync	Validated in Agentic-RL	Validated size	Perf.
Slime / Miles	✗	✗	$\sim$ TB	7–40 s
MoonshotAI / Checkpoint Engine	✗	✗	$\sim$ TB	$\leq$ 20 s
AWex	✗	✗	$\sim$ TB	$\leq$ 10 s
Composer2	✓	✓	$\sim$ TB	?
Pulse	✓	✗	$\sim$ 7 B	?
SparseRL-Sync (ours)	✓	✓	$\sim$ TB	$\approx$ 1 s

Models under study. Where measurements are taken on a single model, we use Qwen3-30B-A3B-Instruct-2507. For the cross-scale study in Figure 7 and throughout the paper, we evaluate four models spanning dense and MoE architectures from 8 B to 671 B parameters: **Qwen3-8B-Base** (abbreviated **8B**), **Qwen3-30B-A3B-Instruct-2507** (**30B**), **GLM-4.5-Air-Base** (**106B**), and **DeepSeek-V3.1-Base** (**671B**). For brevity, we refer to each model by its parameter-count abbreviation in subsequent figures, tables, and prose. We next summarize the main empirical observations that emerge from these measurements.

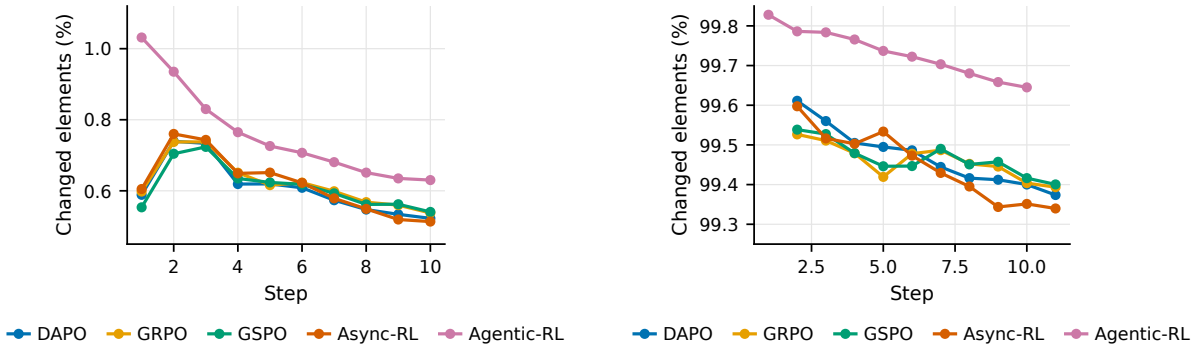
Table 2: Sparsity summary on 30B across five RL settings. BF16 sparsity and inactive tensors are measured at the synchronized working-precision weights; FP32 change ratio is measured on the Trainer-side main weights.

Setting	BF16 sparse	BF16 changed	FP32 changed	Inactive tensors
DAPO	99.4003%	0.5997%	99.4704%	4.9732%
GRPO	99.3859%	0.6141%	99.4595%	5.5216%
GSPO	99.3958%	0.6042%	99.4653%	5.9673%
Async-RL	99.4006%	0.5994%	99.4354%	5.3852%
Agentic-RL	99.3030%	0.6970%	99.6845%	5.4193%

Pervasive element-level sparsity. Table 2 reports the per-step element-level sparsity averaged across an entire training run. Despite the diversity of objective functions, all five settings consistently reach 99.30%–99.40% sparsity, with the last-step sparsity slightly higher than the run-wide mean. The implication is direct: at any synchronization point, fewer than 1% of BF16 weight elements actually need to be transmitted to Rollout.

BF16 vs. FP32: a precision-gated sparsity gap. Table 2 and Figure 4 show that as training progresses, the element-level change ratio of *BF16* weights steadily decreases, dropping from roughly 0.75% in the early steps to below 0.56% by step 8. By contrast, the Trainer-side FP32 *main* weights remain near-dense throughout, with the element-level change ratio consistently above 99.4%. This precision-dependent gap is consistent across all five RL settings.

This behavior is not accidental. Most updates produce micro-changes that fall below the BF16 quantization threshold; they are therefore *absorbed* during the FP32-to-BF16 cast and become invisible after rounding. In other words, the sparsity emerges at the precision-conversion stage rather than in the underlying FP32 update itself. We provide a quantitative explanation



(a) BF16 model parameters.

(b) FP32 master weights.

Figure 4: Precision-gated sparsity gap over synchronization steps. (a) BF16 model parameters synchronized to Rollout have sub-1% changed-element density; (b) FP32 master weights on the Trainer side remain near-dense throughout. Both panels share the same algorithm legend, shown below each subfigure.

of this precision-gated effect in Section 2.2. This precision-gated gap is the foundation of the lossless sparse-sync mechanism described in Section 3.

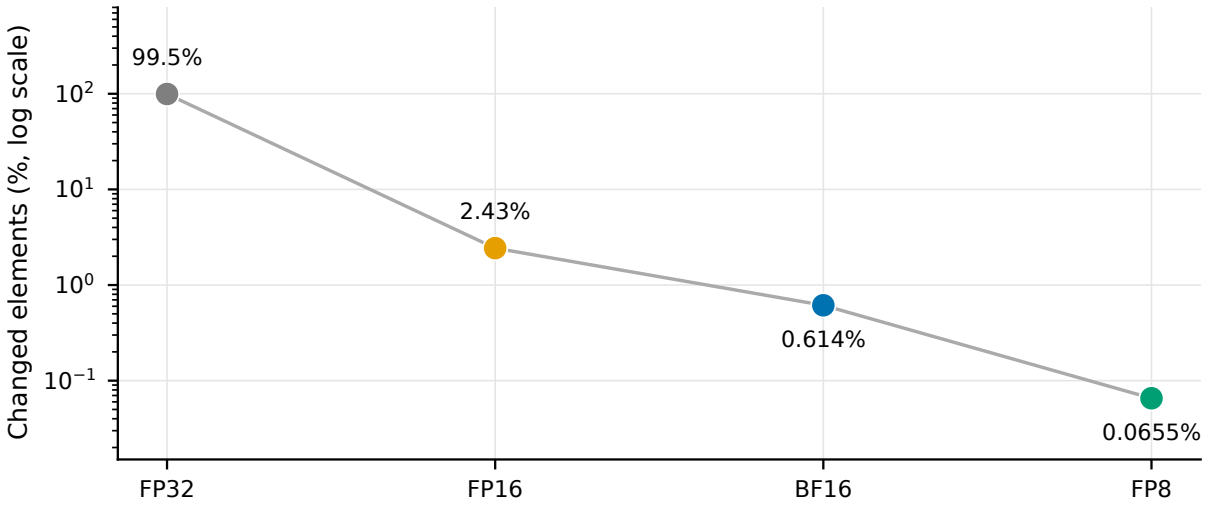


Figure 5: Element-level changed-element density under different synchronization precisions, on a log scale so that the FP16 / BF16 / FP8 differences remain visible. Values shown are measured on Qwen3-30B-A3B over a GRPO run.

Precision controls visible sparsity. Figure 5 compares the apparent sparsity induced by different numerical formats. FP32 main weights have only 0.54% sparsity in the GRPO run, meaning that almost every FP32 element changes after an optimizer update. After casting to BF16, the same synchronization boundary exposes only 0.6141% changed elements. FP16, despite having higher numerical precision than BF16 overall, has a *finer* mantissa (10 bits vs. 7 bits) and therefore absorbs fewer micro-updates at the cast boundary, exposing 2.4308% changed elements and 97.5692% sparsity—more than BF16. FP8 has a coarser visibility threshold and exposes only 0.0655% changed elements, yielding 99.9345% sparsity. These measurements show that the sparse-update opportunity is precision dependent: it is weak in FP32, strong in FP16, stronger in BF16, and strongest in FP8.

Element-level vs. tensor-level sparsity. Table 2 and Figure 6 switch to the parameter-tensor level and report the fraction of tensors for which *no* element changed. Across the five

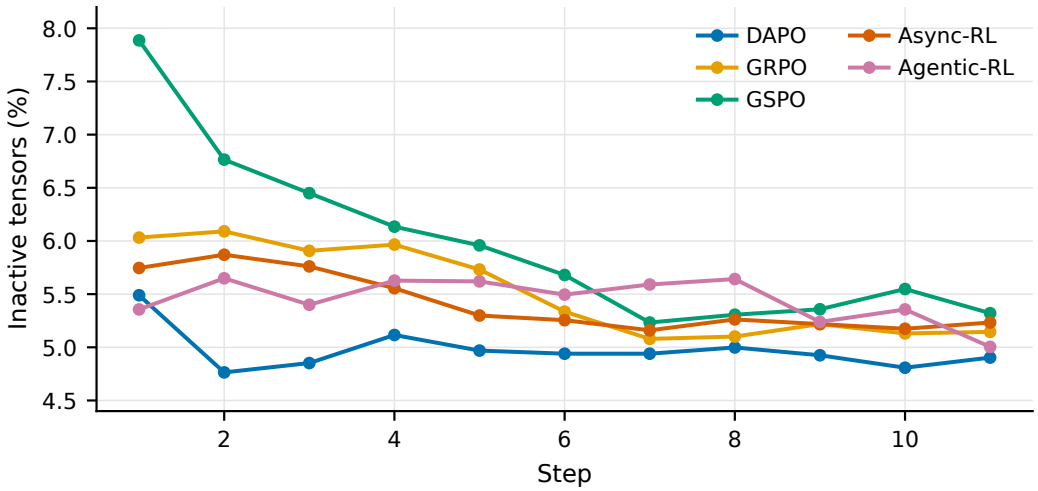


Figure 6: Tensor-level inactive ratio over synchronization steps. Only about 5%–6% of parameter tensors have no changed elements, so the observed sparsity is primarily within tensors.

settings, the inactive-tensor ratio is only about 5%–6%, so more than 94% of tensors still contain at least one changed element at each synchronization point. The sparsity is therefore structural *within* tensors, not between them. This rules out the naive optimization "skip tensors that did not change", and shows that any practical sparse-sync mechanism must operate *at the element level*.

This conclusion also holds for the expert weights of MoE models, where one might suspect that routing concentrates updates on a small subset of experts. On 30B, we tracked the update status of each expert tensor over 10 consecutive synchronization steps: only 14 expert tensors (0.46%) were never updated in any step, while 3,010 expert tensors (97.98%) were updated in *every* one of the 10 steps. Even among experts—the structural component most plausibly amenable to coarse-grained skipping—tensor-level inactivity is a vanishing minority. The sparse-update opportunity is genuinely at the element level.

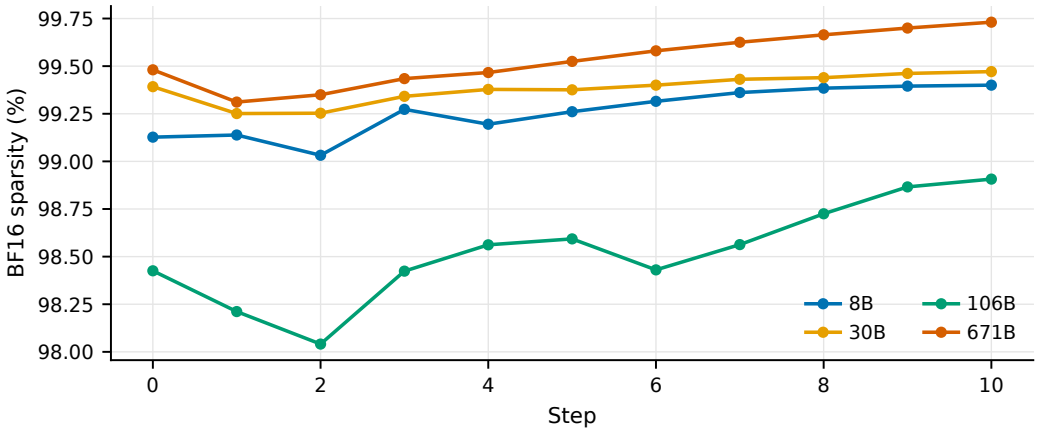


Figure 7: BF16 element-level sparsity across the four model scales (8B, 30B, 106B, 671B) over synchronization steps. All models exhibit high sparsity ($\geq 98\%$) from the first step, and sparsity tends to increase over training. The 671B model reaches the highest observed sparsity, consistent with larger pretrained weights having more mass in the sub-threshold regime that gets absorbed by the BF16 cast.

Sparsity across model scales. Figure 7 extends the sparsity observation to all four model scales (8B, 30B, 106B, 671B). All four models exhibit $\geq 98\%$ BF16 sparsity from the very first synchronization step, confirming that high update sparsity is not a property of any particular model scale or architecture. The 671B model climbs to $\geq 99.5\%$ sparsity within a handful of

steps and stays there, while the 106B MoE model shows somewhat lower sparsity (∼98%–99%), likely reflecting architectural differences in weight magnitude distributions. Across all scales, sparsity tends to increase over training.

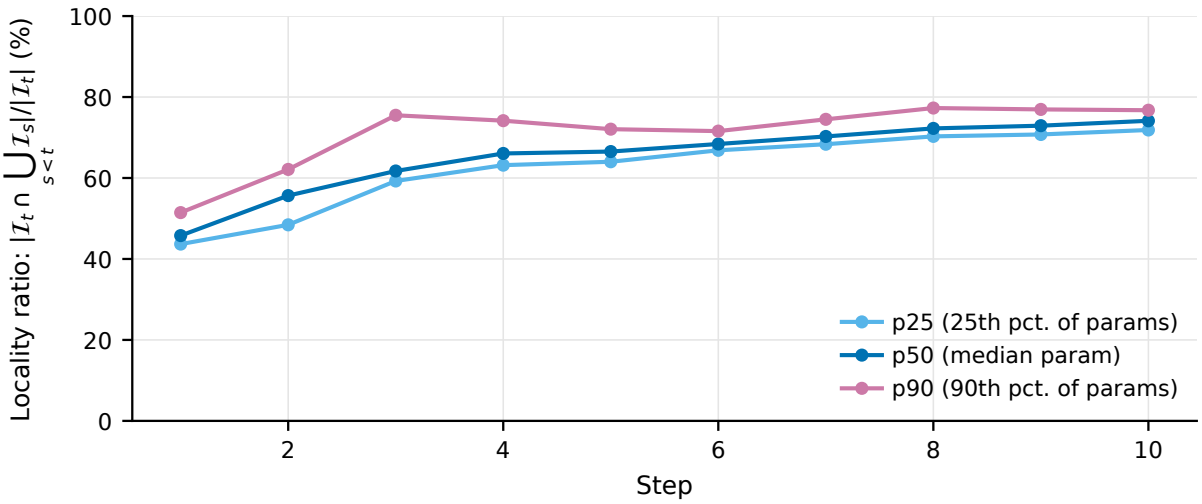


Figure 8: Temporal locality of update indices on 30B (GRPO). For each sync step t and each parameter tensor, we compute the locality ratio $|\mathcal{I}_t \cap \bigcup_{s<t} \mathcal{I}_s| \,/\, |\mathcal{I}_t|$—the fraction of the current changed indices that have appeared in any prior step. The three curves show the 25th, 50th (median), and 90th percentiles of this ratio across all parameter tensors. All three rise monotonically from $\sim 45\%$–52% at step 1 to $\sim 72\%$–77% by step 10, indicating that the majority of updated weight positions recur from earlier synchronization events.

Temporal locality of update indices. Beyond element-level sparsity, the changed indices exhibit strong *temporal locality*: the set of weight elements updated at step t overlaps substantially with the union of indices from all prior steps. Figure 8 quantifies this on 30B (GRPO). For each step t and each parameter tensor, the locality ratio $|\mathcal{I}_t \cap \bigcup_{s<t} \mathcal{I}_s| \,/\, |\mathcal{I}_t|$ measures what fraction of the current changed indices reappear from history. The p25/p50/p90 quantiles across parameter tensors all rise monotonically from $\sim$45%–52% at step 1 to $\sim$72%–77% by step 10. The tight spread between quantiles confirms that this locality is a consistent property across parameter tensors, not driven by a small subset of outlier parameters. This temporal concentration indicates that a persistent "hot" subspace of weight elements accounts for the majority of updates across the training run, and is a promising direction for future delta-index compression schemes.

Takeaways. We summarize the empirical regularities exploited by SparseRL-Sync:

1. **Pervasive element-level sparsity.** Across all RL settings we tested, the BF16 update is $\sim$99.4% sparse on average and tends to become sparser over training.

2. **Precision-gated gap.** The same updates are near-dense in FP32 main weights but highly sparse in BF16 model weights, meaning that the sparsity arises at the FP32-to-BF16 cast.

3. **Precision-dependent visibility.** FP16 exposes more changed elements than BF16 (finer mantissa absorbs fewer micro-updates), while FP8 exposes fewer; the sparsity is a property of the synchronization precision, not only of the optimizer step.

4. **Structural within-tensor sparsity.** Almost every tensor is touched on every step, so naive tensor-skipping is insufficient; element-level indexing is required.

5. **Cross-scale universality.** High BF16 update sparsity ($\geq 98\%$) holds across model scales from 8B to 671B and across dense and MoE architectures.

6. **Temporal locality.** For each parameter tensor, the fraction of its current changed indices that appeared in any prior step rises monotonically from $\sim 45\%$ at step 1 to $\sim 72\%$ by step 10 (median across tensors), indicating a persistent "hot" subspace of frequently updated elements.

2.2 Mechanistic Explanation: The Three-Gate Theory

Zhu et al. (2025) provide a principled explanation for why RL fine-tuning of pretrained large language models yields apparently sparse parameter updates. We adopt their Three-Gate theory here as the explanatory framework for our observations, not as a new theoretical contribution of our own, but as a compact mechanistic account of why the BF16 weight delta synchronized from Trainer to Rollout is highly sparse across the RL settings we study. This framework directly motivates both the design of SparseRL-Sync (Section 3) and the ablation experiments used to test it (Section 4).

As summarized in Figure 9, the apparent sparsity of RL weight updates arises from three coupled effects: a KL-constrained small-step regime, geometry-induced routing away from dominant pretrained directions, and BF16 rounding that suppresses many resulting micro-updates. Together, these effects explain why the underlying FP32 update can remain nearly dense while the BF16 weight delta seen by Rollout becomes highly sparse. The conclusion we draw—and that underlies SparseRL-Sync—is that the $\sim 99\%$ sparsity we observe is not an artifact of any single RL algorithm or model family, but a structural consequence of the small-step, low-precision regime in which contemporary large-model RL typically operates.

3 Method: SparseRL-Sync

Building on the empirical pervasiveness of element-level update sparsity (Section 2.1), we now present SparseRL-Sync, a *lossless* sparse-synchronization mechanism that replaces full-weight synchronization with (indices, values) messages reconstructible bit-for-bit on the Rollout side. Throughout this section we use *master weights* (W^{main}) for the high-precision parameter copy maintained by the optimizer (e.g., FP32 in standard mixed-precision training) and *model parameters* (W) for the working-precision copy used in forward and backward passes (e.g., BF16).

3.1 Design Goals and Overview

Design goals. SparseRL-Sync is engineered to satisfy four properties:

G1. Lossless fidelity. Rollout receives *exactly* the model weights the Trainer has, so the policy used for sampling is identical to the one being trained.

G2. Drop-in integration. The Trainer-side change is local to the optimizer-step boundary; the Rollout-side change is local to the weight-loader.

G3. Bandwidth reduction $\propto$ sparsity. The on-wire payload should scale as $\Theta(|\mathcal{I}|)$ in the number of changed elements.

G4. Universality. One mechanism covers dense / MoE models, full fine-tuning / LoRA.

Overall workflow. Recall Figure 2 from Section 1: the right column depicts the SparseRL-Sync pipeline and contrasts it with the full-update baseline on the left. Four additional steps (highlighted in vermillion) are inserted into the existing pipeline:

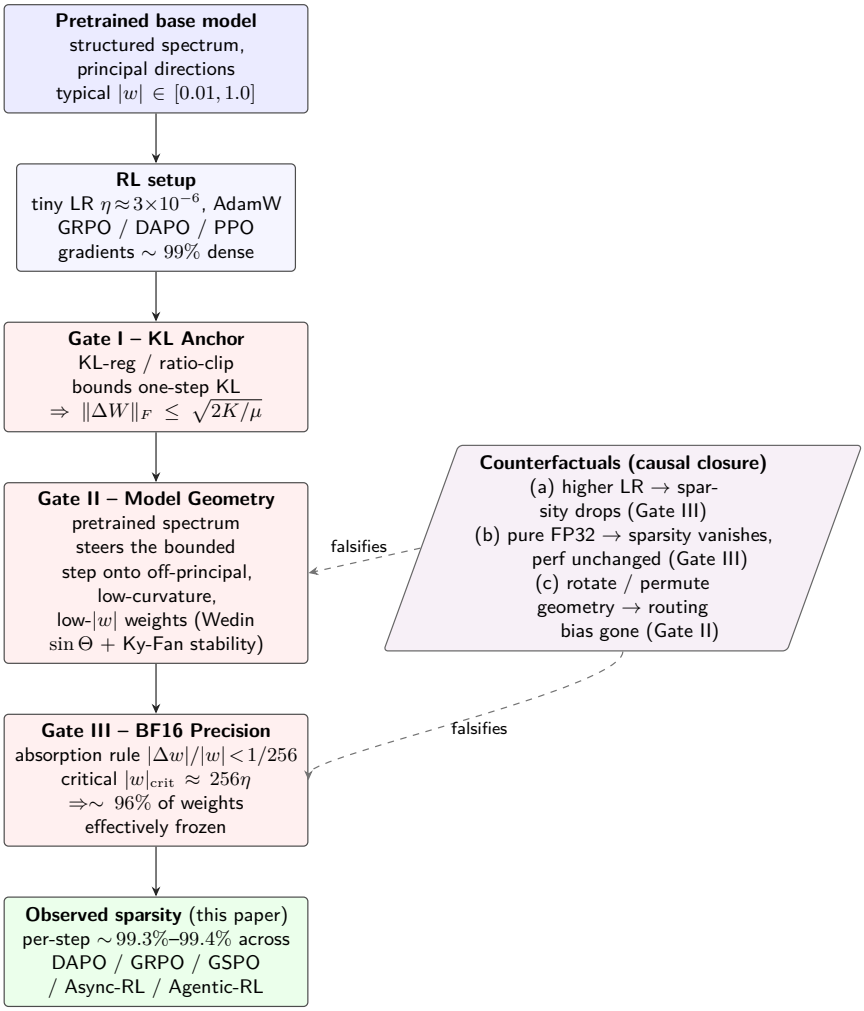


Figure 9: Three-Gate Theory of Zhu et al. (2025), reproduced here as the explanatory framework for our sparsity observations. The pretrained base model and the RL optimizer (top) jointly set a small-step, KL-bounded regime; three successive gates—Gate I (KL anchor) bounds step magnitude, Gate II (model geometry) routes the bounded update onto off-principal, low-curvature coordinates, and Gate III (BF16 precision) suppresses sub-threshold updates at the FP32→BF16 cast—together account for the $\sim 99\%$ element-level sparsity we measure. The box on the right lists counterfactual probes from Zhu et al. (2025) that isolate the contribution of individual gates.

1. *Compute indices.* After the master weights are cast and copied into the model parameters, the Trainer compares the pre- and post-copy values elementwise to obtain the set of changed indices.

2. *Materialize a change-masked tensor.* Using these indices, the Trainer constructs a parameter-shaped tensor initialized with a sentinel value (NaN), then writes the current model-parameter values at the changed positions. Unchanged positions remain as NaN and carry no information.

3. *Convert to* (I, V). Before transmission, the sparse tensor is converted into an (indices, values) payload (optionally losslessly encoded; Section 3.3).

4. *Reconstruct on Rollout.* Each Rollout rank applies the received (I, V) updates as an in-place scatter into its local weight buffer.

The control plane is unchanged: synchronization events are still triggered via Ray remote calls, and the underlying parameter broadcast uses the same PyTorch process groups (NCCL) as

Algorithm 1 Trainer: optimizer step with index tracking

Require: Model parameters W (e.g., BF16); master weights W^{main} (e.g., FP32); steps $1{:}T$.
Ensure: Updated W and the cumulative changed-index set $\mathcal{I}_T$.

$\mathcal{I}_0 \leftarrow \emptyset$
for $t \leftarrow 1$ **to** T **do**
  OptimizerStep(t) ▷ updates W^{main}
  $W_{\text{prev}} \leftarrow \text{detach\_clone}(W)$
  CastAndCopy($W \leftarrow \text{round}_{\text{BF16}}(W^{\text{main}})$)
  $I_t \leftarrow \{\, i \mid W^{(i)} \neq W^{(i)}_{\text{prev}} \,\}$ ▷ precision filter: working-precision cast absorbs sub-threshold updates
  $\mathcal{I}_t \leftarrow \mathcal{I}_{t-1} \cup I_t$
end for
return $(W, \mathcal{I}_T)$

Algorithm 2 WeightUpdater: pack and broadcast sparse updates

Require: Named tensors $\{(name, param, \mathcal{I})\}$ with changed-index set $\mathcal{I}$.
Ensure: Sparse update message delivered to Rollout.

meta $\leftarrow [\,]$; *I_list* $\leftarrow [\,]$; *V_list* $\leftarrow [\,]$
for all $(name, param, \mathcal{I})$ **do**
  $M \leftarrow$ Materialize($param$, $\mathcal{I}$) ▷ param-shaped; NaN at unchanged positions, $param^{(i)}$ at $i \in \mathcal{I}$
  $U \leftarrow$ ConvertForBroadcast(M) ▷ gather complete parameter tensor, fix layout and dtype
  $\mathcal{V} \leftarrow U[\mathcal{I}]$ ▷ $\mathcal{I}$:`int32`, $\mathcal{V}$:`BF16`
  $(\mathcal{I}, \mathcal{V}) \leftarrow$ OptionalEncode($\mathcal{I}, \mathcal{V}$) ▷ see Section 3.3
  append $(name, \text{dtype}(U), \text{shape}(U))$ **to** *meta*
  append $\mathcal{I}$ **to** *I_list*; **append** $\mathcal{V}$ **to** *V_list*
end for
SendToRollout(*meta*, *I_list*, *V_list*)

before; the only structural difference is that each bucket's on-wire size is reduced in proportion to the element-level update sparsity.

3.2 Algorithms

The mechanism is implemented by three cooperating procedures: (1) the Trainer collects changed indices alongside its optimizer step (Algorithm 1); (2) a WeightUpdater module packs the indices and values into a self-describing message (Algorithm 2); (3) each Rollout rank applies the message in place (Algorithm 3).

Algorithm 1 accumulates element-level deltas across T optimizer steps. The cumulative set $\mathcal{I}_T$ is the set of model-parameter elements that need to be communicated at the next synchronization point, and is by construction *precision-aware*: indices that vanish at the precision-reducing cast from W^{main} to W are never inserted. Note that $\mathcal{I}_T$ is a conservative *superset* of the true post-T delta against the last synchronization snapshot: an element that changed at some intermediate step but whose final value equals the pre-sync value still appears in $\mathcal{I}_T$. This overapproximation is harmless for correctness (Algorithm 2 transmits the current values $W[\mathcal{I}_T]$, so Rollout receives a bit-exact copy of W regardless of redundant indices) and only costs a bounded amount of extra bandwidth proportional to the superset size. The snapshot W_{prev} is freed immediately after I_t is computed, so this step adds no persistent memory overhead.

Algorithm 2 packs every changed parameter into a self-describing payload (*meta*, *I_list*, *V_list*).

Algorithm 3 Rollout: receive and apply sparse updates

Require: Local weights W.
Ensure: Updated local weights $\widetilde{W}$.

```
(meta, I_list, V_list) ← RecvFromUpdater( )
for k ← 1 to |meta| do
    (name, dtype, shape) ← meta[k]
    I ← I_list[k];   V ← V_list[k]
    (I, V) ← OptionalDecode(I, V)
    W[name][I] ← V                          ▷ in-place sparse scatter
end for
return W̃
```

Indices are kept in `int32` (so the encoding is correct for any tensor shape that fits in 2^{31} flattened elements, i.e. all parameter tensors in current open large models including DeepSeek-V3.1-Base). ConvertForBroadcast assembles the complete parameter tensor from its distributed shards and applies any required layout and dtype conversion; values $\mathcal{V}$ are then extracted from this fully-assembled tensor, so the receiver does not need any knowledge of the Trainer's parallelism layout. The optional encode step, Section 3.3, is where additional bandwidth gains compound on top of the raw (I, V) cost.

Algorithm 3 mirrors Algorithm 2 on the Rollout side. The optional decode is the exact inverse of the optional encode, and the sparse scatter $W[name][\mathcal{I}] \leftarrow \mathcal{V}$ is implemented as a fused kernel. Because the values transmitted are the bit-exact post-cast model-parameter values, after this step the local Rollout weights are bit-identical to the Trainer's model parameters—this is the property G1 (lossless fidelity).

3.3 Cost Model and Lossless Compression

Raw (I, V) cost model. Let $S = N \cdot b_v$ be the size in bytes of a full working-precision weight broadcast (N elements at b_v bytes each), and let $\rho \in [0, 1]$ be the element-level update density (so $1 - \rho$ is the sparsity). Encoding indices as fixed-width `int32` integers ($b_i = 4\,\mathrm{B}$) alongside BF16 values ($b_v = 2\,\mathrm{B}$), the sparse payload has size

$$S_{\text{sparse}}(\rho) \;=\; \rho N\,(b_v + b_i) \;+\; S_{\text{meta}}, \tag{1}$$

where S_{meta} is the constant per-tensor metadata overhead. Ignoring S_{meta}, the raw compression ratio is

$$X(\rho) \;=\; \frac{S}{S_{\text{sparse}}(\rho)} \;\approx\; \frac{b_v}{\rho\,(b_v + b_i)} \;=\; \frac{1}{3\rho}. \tag{2}$$

Lossless compression of the (I, V) payload. The raw formula of Equation (2) treats the indices and values as unstructured binary blobs. We apply two complementary lossless transforms before transmission and their exact inverses on the receiver, preserving G1 (lossless fidelity) exactly.

Index delta encoding. Because the changed indices $\mathcal{I}$ are sorted, we store the first-differences $\Delta\mathcal{I}$ (prepending a zero) instead of absolute values. Empirically, the maximum inter-index gap for typical linear-weight tensors is well below 2^{15} (measured maxima are in the low thousands), so the deltas fit in `int16` ($b_i = 2\,\mathrm{B}$), halving the index stream. Embedding-table tensors, whose indices can span up to $\sim 10^8$ positions, retain `int32` ($b_i = 4\,\mathrm{B}$).

Value entropy coding. The BF16 value stream $\mathcal{V}$ is also highly compressible: changed weight values cluster near their previous magnitudes, yielding a distribution that standard lossless entropy coders exploit well. Empirically, entropy coding reduces the value stream to $\alpha \in [0.60, 0.70]$ of its original size.

Combining both passes, the compressed payload size becomes

$$S_{\text{compressed}}(\rho) \;=\; \rho N\,(b_i + \alpha\, b_v), \tag{3}$$

where $b_i = 2\,\text{B}$ (delta-encoded `int16` for linear weights) or $4\,\text{B}$ (embedding tables), and the combined compression ratio is

$$X_{\text{compressed}}(\rho) \;=\; \frac{b_v}{\rho\,(b_i + \alpha\, b_v)}. \tag{4}$$

A concrete example reaching $\approx 100\times$. The compression ratio in Equation (4) depends only on ρ, b_v, b_i, and α—not on the absolute model size. We illustrate with 30B, which reaches a mean element-level sparsity of 99.38% over GRPO fine-tuning (Figure 7), corresponding to an update density of $\rho = 0.62\%$. Using delta-encoded `int16` indices ($b_i = 2\,\text{B}$) and value entropy coding at $\alpha = 0.60$, the combined ratio is

$$X_{\text{compressed}}(0.0062) \;=\; \frac{2}{0.0062\,(2 + 0.60 \times 2)} \;\approx\; 100\times.$$

Equivalently, each changed element occupies $b_i + \alpha b_v = 3.2\,\text{B}$ on the wire versus $b_v = 2\,\text{B}$ for every element in the full-update baseline, so the per-parameter sparse cost is 1.6ρ of the full-update cost. Across the four model scales (8B–671B) reported in Figure 1a, ρ remains below 1.1%, yielding raw (I, V) ratios of $32\times$–$54\times$ and compressed ratios of $60\times$–$101\times$.

3.4 Integration

Trainer. The hook sits at the optimizer-step epilogue, between the high-precision optimizer update to the master weights and the cast-and-copy step that materializes them into model parameters (CastAndCopy in Algorithm 1). Immediately after this step writes the new model-parameter values, we diff the new and previous parameter buffers to collect $\mathcal{I}_t$. This change is local to the optimizer-step boundary and is independent of the parallelism backend.

WeightUpdater. Triggered by the RL framework via a Ray remote call at each synchronization event. The WeightUpdater iterates over named parameters and routes each one based on its update density: parameters with high sparsity follow the (I, V) path of Algorithm 2; parameters that change on nearly every element each step are transmitted via a full-weight copy. For example, LoRA adapter weights—which are small by design and exhibit near-100% update density—take the full-copy path, while frozen base-model parameters exhibiting $\geq 99\%$ sparsity take the sparse path. The routing is per-parameter and transparent to the training loop.

Rollout. The hook patches the weight-loader's `tensor.copy_()` call. Instead of copying a full parameter tensor, the patched copy accepts an (I, V) payload, unpacks it into a NaN-masked buffer, and applies an in-place sparse scatter (Algorithm 3). The same patch applies to any inference framework that loads weights through a parameter copy step.

4 Experiments

Our evaluation answers two questions:

1. **Correctness** (Section 4.2): does sparse (I, V) reconstruction preserve the RL trajectory bit-exactly, and is the reward curve indistinguishable from a full-update baseline?
2. **Communication savings** (Section 4.3): how much does SparseRL-Sync reduce the on-wire payload and transmission time across model scales and bandwidth regimes?

4.1 Setup

Framework and models. All runs use **Helix** (Scitix, 2026), our in-house RL framework, with Megatron-LM for the Trainer and SGLang for the Rollout. Integration follows Section 3.4: a Trainer-side hook at the optimizer cast boundary (Algorithm 1) and a Rollout-side patch at the weight-loader boundary (Algorithm 3). We use 30B for the correctness study, and report communication savings in the TB-scale regime where bandwidth is the dominant cost: 106B (measured) and 671B (projected).

Hardware. Each node is a single-socket 8×H100-SXM5 server with 4 RDMA NICs (one NIC per two GPUs).

Bandwidth regimes. Point-to-point GPU–GPU benchmarks from one 8-GPU node to 15 peers define two inter-node regimes used throughout:

- **IB on** (RDMA active): per-GPU 34.99 GB/s mean (30.3–37.9); per-node ≈280 GB/s.
- **IB off** (TCP fallback): per-GPU 2.84 GB/s mean (0.91–5.30); per-node ≈22.7 GB/s.

Intra-node (NVLink) is ≈319 GB/s per GPU. The two inter-node regimes bracket the deployment envelope from well-provisioned RDMA clusters to cross-cluster / TCP-only settings, where sparse synchronization matters most.

4.2 Correctness Validation

Bit-exact reconstruction. At each synchronization event the Rollout first applies the full-weight update and snapshots the result, then restores the previous state and applies the sparse (I, V) update. Comparing the two copies tensor by tensor, all layers match bit-for-bit on every synchronization event across the full run.

Reward validation. Figure 1b and Table 3 compare rollout-level rewards of a full-update baseline and a SparseRL-Sync run on 30B over 500 steps. The curves are visually indistinguishable: mean-reward diff -8×10^{-6}, MAE 0.0186, per-rollout Pearson correlation 0.9749.

Table 3: Reward-level validation on 30B over 500 steps.

Metric	Full-update mean	SparseRL-Sync mean	Diff.	MAE	Corr.
Reward	0.738095	0.738087	-8×10^{-6}	0.0186	0.9749

These results confirm that SparseRL-Sync does not measurably perturb the sampled reward trajectory.

4.3 Communication Savings Across Model Scales

Setup. We measure 106B on 128 H100 GPUs in separated (disaggregated) mode, split evenly as 64 Trainer + 64 Rollout. A separated 671B deployment would require more than 128 GPUs, so we instead *project* its broadcast time by applying the 106B effective bandwidth to the 671B full-update payload (1 342 GB) and sparse payload (≈31.0 GB at $\rho \approx 0.77\%$). The projection is conservative: larger-scale collective broadcasts typically achieve equal or better utilization than the 106B baseline. All SparseRL-Sync numbers in this subsection use the raw (I, V) path; the additional lossless index/value compression of Section 3.3 would shrink the SparseRL-Sync payloads further, which we leave to future measurements.

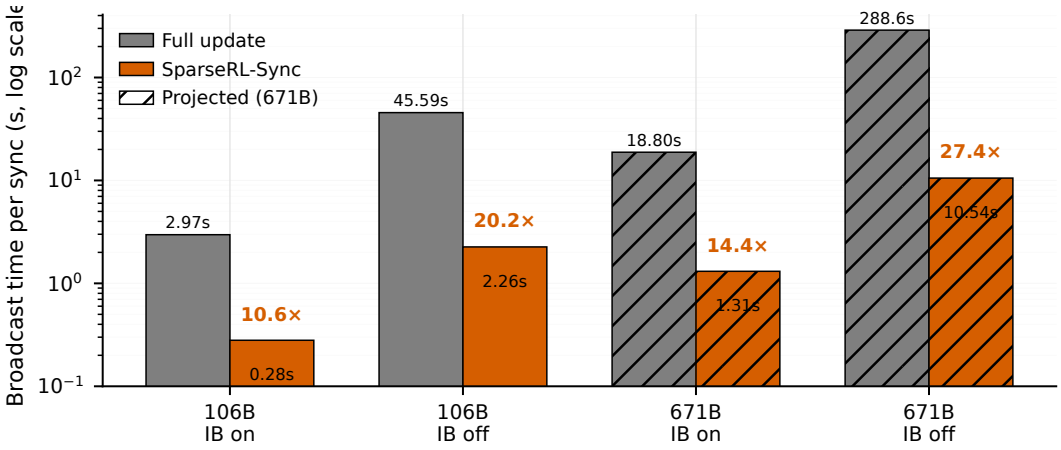


Figure 10: Per-synchronization broadcast time under the two bandwidth regimes of Section 4.1. 106B is measured on 128 H100 GPUs in separated mode (64 Trainer + 64 Rollout); 671B is projected from the 106B effective bandwidth (hatched bars). Numbers on top of each SparseRL-Sync bar are speedups over the corresponding full-update baseline. Note the log-scale y-axis.

Findings. Three observations stand out. **(i) IB-off becomes usable.** Without RDMA, a full 106B broadcast takes 45.6 s; SparseRL-Sync brings it down to 2.26 s—matching the single-digit budget that the full-update path only achieves with RDMA. The same pattern projects to 671B: nearly 5 minutes collapses to $\approx$10.5 s. **(ii) Speedup grows as bandwidth shrinks.** We observe 10.6×–14.4× under IB on and 20.2×–27.4× under IB off, consistent with the cost model of Section 3.3: once the payload is an order of magnitude smaller, its transmission time depends far less on the bandwidth gap between regimes. **(iii) Sync cadence improves by an order of magnitude.** The 106B IB-off drop from 45.59 s to 2.26 s moves Trainer $\rightarrow$ Rollout synchronization from "many-second" to "low-second," the threshold at which Async-RL deployments no longer need to hide synchronization behind rollout tail latency.

5 Related Work

SparseRL-Sync is most closely related to three lines of prior work. The first is the algorithmic literature on RL fine-tuning of large language models, which defines the optimization regime in which our system operates. The second is the systems literature on weight synchronization and gradient/weight compression for distributed training, which addresses similar bandwidth bottlenecks and provides technical building blocks that we re-use. The third is the recent literature on the parameter-space dynamics of RLVR, which provides the mechanistic basis for our sparsity observations.

5.1 RL Fine-Tuning of Large Language Models

Reinforcement learning from human feedback (Ouyang et al., 2022) and Proximal Policy Optimization (PPO) (Schulman et al., 2017) laid the foundation for RL-based LLM post-training. In long-horizon reasoning settings, however, PPO's reliance on a learned value function can introduce substantial credit-assignment and optimization overhead (Kazemnejad et al., 2024), motivating critic-free alternatives such as Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which estimates advantages from multiple rollouts per prompt.

A central challenge in this line of work is the stability of importance-ratio and advantage-based updates. DAPO (Yu et al., 2025) introduces decoupled clipping and dynamic sampling for large-scale LLM RL; SAPO (Gao et al., 2025) replaces hard clipping with a smooth, adaptive gate; and GSPO (Zheng et al., 2025) moves importance-ratio control from the token level to the sequence level. Related work further studies off-policy or sample-polarity effects, including

tapered off-policy REINFORCE (Le Roux et al., 2025), asymmetric importance-sampling correction (Wang et al., 2025), adaptive advantage shaping (Tang et al., 2025), and negative-enhanced GRPO (Nan et al., 2025).

SparseRL-Sync is *algorithm-agnostic*: it operates on the post-cast BF16 weight delta and does not modify the loss, the importance ratio, or the optimizer. We therefore treat these algorithms as benchmark settings (Section 2.1) rather than as competitors.

5.2 Weight Synchronization and Compression in Distributed Training

The communication cost of Trainer-to-Rollout weight synchronization is determined by model size, synchronization frequency, and the network conditions between Trainer and Rollout. As RL deployments move toward larger models and increasingly disaggregated resource pools, this path has become a first-class systems concern. Recent systems have therefore optimized the weight-update path in several complementary ways.

Engineering optimized full-weight systems. A first line of systems work focuses on making *full*-weight transfer practical through engineering optimizations:

- *Slime/Miles* (THU-DCST, 2024) provides an open-source RL post-training framework that connects high-performance training with rollout/inference backends. Its full-update Trainer-to-Rollout weight-update pipeline serves as the baseline path illustrated in Figure 2 (left column).
- *Kimi checkpoint engine* (Moonshot AI, 2024) reports efficient in-place weight updates for a 1T-parameter Kimi-K2 model across thousands of GPUs in approximately 20 seconds.
- *AWex* (Ant Group / inclusionAI, 2024) is a high-performance RL training–inference weight-synchronization framework designed to enable second-level parameter updates from training to inference, with support for heterogeneous deployment modes and transfer optimizations.

These systems substantially reduce synchronization latency, but they leave the underlying communication object unchanged: the payload is still a full BF16 or quantized weight tensor, so the transmitted data volume scales with model size rather than with the number of elements that actually changed.

Sparse and delta-based systems. A second line of work reduces the communicated payload by sending deltas or sparse updates:

- *Composer2* (Cursor, 2024) describes a geographically distributed RL infrastructure in which inference clusters reconstruct weights from a shared delta chain over commodity cloud storage. The public report describes the high-level delta-chain design, but does not fully disclose the sparse encoding details or provide a public implementation of the synchronization layer.
- *PULSE* by Miahi and Belilovsky (Miahi and Belilovsky, 2026) performs the closest analysis of weight-update sparsity in distributed RL and proposes a lossless sparse update path that transmits the indices and values of modified parameters. Their public evaluation focuses on bandwidth-constrained decentralized RL settings, whereas SparseRL-Sync targets MoE models, TB-scale synchronization, and integration with production training stacks.

SparseRL-Sync extends this line of work along three axes: (a) coverage of MoE architectures up to 671B parameters, (b) support for both Megatron-LM and FSDP-style training stacks, and (c) support for both full fine-tuning and LoRA. We also release an open-source implementation integrated with `slime` and SGLang.

Communication-efficient distributed SGD. A long line of work studies the more general problem of compressing *gradients* in data-parallel SGD, including 1-bit SGD (Seide et al., 2014), TernGrad (Wen et al., 2017), top-k sparsification (Aji and Heafield, 2017), Deep Gradient Compression (Lin et al., 2018), and PowerSGD (Vogels et al., 2019). Our setting differs in two crucial ways. First, we communicate rounded model weights rather than gradients, so we do not require error feedback to compensate for dropped gradient information. Second, we require lossless reconstruction of the inference-visible weights, which rules out lossy compressors such as sign-based, top-k, or low-rank approximations. We therefore reuse only the encoding tools from this literature, such as compact index encodings and entropy coding, while avoiding lossy approximation of the communicated values.

5.3 Parameter-Space Dynamics of RLVR

A complementary line of work asks not *how* to compress RL updates, but *why* they appear sparse in the first place. Zhu et al. (2025) provide a parameter-level account of RLVR training and propose the Three-Gate Theory summarized in Section 2.2: a KL-constrained small-step regime, pretrained model geometry, and low-precision rounding jointly make the post-cast weight delta appear sparse. They also contrast this regime with supervised fine-tuning, which follows different parameter-space dynamics. Our paper takes this explanation as an external theoretical account and operationalizes it as a systems-level design invariant in the RL settings we study: because the BF16 synchronization delta is consistently sparse, we can build a lossless sparse-update infrastructure around it.

Asynchronous and heterogeneous RL. Decoupling Trainer and Rollout is itself an active systems direction. AReaL (Anonymous, 2024a) explores fully asynchronous RL, in which generation is decoupled from training to improve GPU utilization. Composer2 (Cursor, 2024) demonstrates geographically distributed rollout infrastructure across multiple clusters. ROLL (Anonymous, 2024b) provides a large-scale RL framework for LLM training over large GPU resources and heterogeneous training scenarios. These systems make efficient Trainer–Rollout synchronization increasingly important. SparseRL-Sync is complementary to them: rather than changing the RL algorithm or deployment topology, it reduces the synchronized payload itself.

6 Conclusion

We presented SparseRL-Sync, a lossless sparse-synchronization mechanism for the Trainer-to-Rollout weight path in large-model reinforcement learning. We observe that the post-cast model-weight delta synchronized to Rollout is highly sparse at the element level across mainstream RL settings, meaning that full-weight transfer contains substantial redundant communication. Based on this observation, SparseRL-Sync replaces full-weight broadcasts with sparse (I, V) messages that transmit only changed indices and their updated values while preserving bit-exact equivalence to full-update synchronization. Our implementation integrates with Megatron-LM, FSDP, `slime`, and SGLang, supports dense and MoE models as well as full fine-tuning and LoRA, and reduces weight-synchronization volume by 32×–54× at the raw (I, V) level and 60×–101× with lossless index/value compression across model scales from 8B to 671B.

References

Alham Fikri Aji and Kenneth Heafield. Sparse communication for distributed gradient descent. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017.

Anonymous. AReaL: Towards fully asynchronous reinforcement learning for large language models, 2024a. TODO: confirm citation key and arXiv id.

Anonymous. ROLL: Heterogeneous reinforcement learning for large models, 2024b. TODO: confirm citation key and arXiv id.

Ant Group / inclusionAI. AWex: Asynchronous weight exchange for large-model RL training. GitHub repository, 2024. URL https://github.com/inclusionAI/asystem-awex.

Cursor. Composer2: Multi-cluster RL training at Cursor. Technical report, 2024. TODO: replace with the canonical URL once published.

Gao et al. SAPO: Soft asymmetric policy optimization, 2025. TODO: confirm citation; sigmoid-based soft gating.

Amirhossein Kazemnejad et al. VinePPO: Unlocking RL potential for llm reasoning through refined credit assignment, 2024. TODO: confirm citation; cited in info.md as Kazemnejad et al., 2024.

Nicolas Le Roux et al. TOPR: Tapered off-policy REINFORCE for stable off-policy learning, 2025. TODO: confirm citation.

Yujun Lin, Song Han, Huizi Mao, Yu Wang, and William J. Dally. Deep gradient compression: Reducing the communication bandwidth for distributed training. In *International Conference on Learning Representations (ICLR)*, 2018.

Erfan Miahi and Eugene Belilovsky. Understanding and exploiting weight update sparsity for communication-efficient distributed RL, 2026. URL https://arxiv.org/abs/2602.03839.

Moonshot AI. Kimi checkpoint engine. GitHub repository, 2024. URL https://github.com/MoonshotAI/checkpoint-engine.

Nan et al. NGRPO: Negative-aware group relative policy optimization, 2025. TODO: confirm citation.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 2022.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Scitix. Helix: An RL training framework. GitHub repository, 2026. URL https://github.com/scitix/helix. Repository to be released; placeholder URL.

Frank Seide, Hao Fu, Jasha Droppo, Gang Li, and Dong Yu. 1-bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs. In *Fifteenth Annual Conference of the International Speech Communication Association (INTERSPEECH)*, 2014.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. Introduces Group Relative Policy Optimization (GRPO).

Tang et al. A3PO: Adaptive advantage shaping for policy optimization, 2025. TODO: confirm citation.

THU-DCST. slime: an open-source framework for large-model reinforcement learning. GitHub repository, 2024. URL https://github.com/THU-DCST/slime. TODO: confirm canonical citation and version commit.

Thijs Vogels, Sai Praneeth Karimireddy, and Martin Jaggi. PowerSGD: Practical low-rank gradient compression for distributed optimization. In *Advances in Neural Information Processing Systems*, 2019.

Wang et al. ASPO: Asymmetric importance-ratio correction for policy optimization, 2025. TODO: confirm citation.

Wei Wen, Cong Xu, Feng Yan, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. TernGrad: Ternary gradients to reduce communication in distributed deep learning. In *Advances in Neural Information Processing Systems*, 2017.

Yu et al. DAPO: An open-source LLM reinforcement learning system at scale, 2025. TODO: confirm full author list and arXiv id.

Zheng et al. GSPO: Sequence-level group sequence policy optimization, 2025. TODO: confirm citation.

Hanqing Zhu, Zhenyu Zhang, Hanxian Huang, DiJia Su, Zechun Liu, Jiawei Zhao, Igor Fedorov, Hamed Pirsiavash, Zhizhou Sha, Jinwon Lee, David Z. Pan, Zhangyang Wang, Yuandong Tian, and Kai Sheng Tai. The path not taken: RLVR provably learns off the principals, 2025. URL https://arxiv.org/abs/2511.08567.